

第三次课后作业（树与二叉树+优先队列）

题量: 1 满分: 100

智能分析

100分

作答时间: 2025-03-21 17:25 至 2025-04-01 23:00

一. 简答题（共1题，100分）

1. (简答题)

AI讲解

1. 【证明题】定义满二叉树的每一个结点或者是一个分支结点，并恰好有两个非空子结点；或者是叶结点。定义一个结点的度为其非空子结点的数目。作为满二叉树定理的推论，证明任何一棵二叉树中度为2的结点的数目为叶结点数减1。

2. 【算法设计题】

树中两个结点间的路径长度，是指从一个结点沿着树中的边走到另一个结点所经过的最少的边数。例如图5-27所示二叉树中结点D和G之间的路径长度就是4。二叉树的直径是指任意两个结点之间路径长度的最大值。例如图5-27所示二叉树的最长路径在E和G之间，所以直径是5。请给出计算二叉树直径的算法。

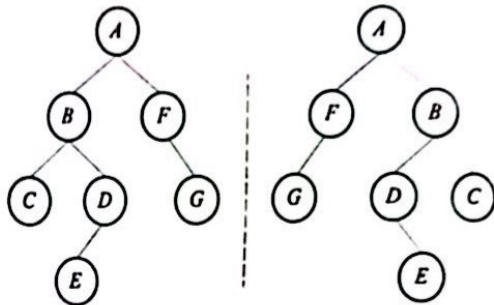


图 5-27 镜像对称的二叉树示例

要求:

- (1) 用自然语言简要描述算法思想;
- (2) 用类C或C++语言或伪代码描述算法步骤;
- (3) 分析算法的时间复杂度。

3. 【应用题】

画出对下列存储于数组中的值执行buildHeap后得到的最大堆:

10 15 12 13 2 11 8 7 9 4

4. 【应用题】

(1) 根据下面给定的字母和权重建立Huffman编码树，并用表格形式给出各个字母的代码；

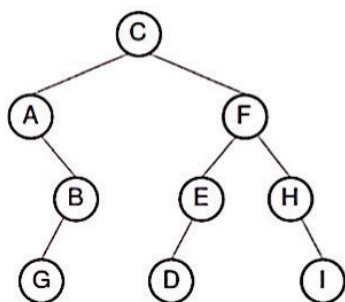
(2) 计算平均码长。

(A, 2) (B, 3) (C, 5) (D, 7) (E, 11) (F, 13) (G, 17)

(H, 19) (I, 23) (J, 31) (K, 37) (L, 41)

5. 【算法设计题】给定二叉树层序序列（定义见教材181页），请设计算法查找二叉树中结点数目最多的层。

例如下图二叉树中结点数目最多的层为3。



要求：

(1) 用自然语言简要描述算法思想；

(2) 用类C或C++语言或伪代码描述算法步骤；

(3) 分析算法的时间复杂度。

1. 证明任一树 T 中 $n_0 = n_1 + 1$ 给定数目

证: 记 n_0 为叶结点数, n_2 为度为2结点数, n_1 为度为1的结点数

即证 $n_2 = n_0 - 1$.

结点总数为 $n_0 + n_1 + n_2 = n$

除根结点处, 其余结点都有1个分支进入, 设 N 为分支总数

$$\begin{cases} n = N + 1, & \text{分支由度为1和2的结点引出.} \\ N = n_1 + 2n_2. \end{cases}$$

$$\Rightarrow n_0 + n_1 + n_2 = n_1 + 2n_2 + 1$$

$$\Rightarrow n_0 = n_2 + 1 \text{ 即 } n_2 = n_0 - 1 \text{ 得证.}$$

2.

(1) 算法设计思想:

使用 DFS 方法递归遍历每个结点, 计算其左右子树的高度。每个结点的左右子树高度之和加2为可能的直径, 维护一个变量记录最大直径遍历完成后, 该最大值为二叉树直径。

(2) 代码

`int height(TreeNode* node, int &maxDia) // 计算可能的最大直径`

`{ if(!node) return -1;`

`int left = height(node->left, maxDia);`

`int right = height(node->right, maxDia);`

`maxDia = max(maxDia, left + right + 2); // 将子树高度转化为边数之和`

`return max(left, right) + 1; // 返回当前结点高度`

`}`

`int diameterOfTree(TreeNode* root)`

`{ int maxDia = 0;`

`if (root) height(root, maxDia); // 递归计算最长直径.`

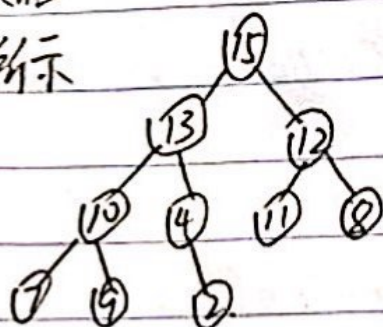
`return maxDia;`

`}`

(3) 时间复杂度: $O(n)$. 算法通过 DFS 遍历二叉树每个结点, 对每个结点递归计算左右子树高度并更新直径, 为 $O(1)$ 操作, 共有 n 个结点, 故总体时间复杂度为 $O(n)$.

3. 最大堆

如图所示



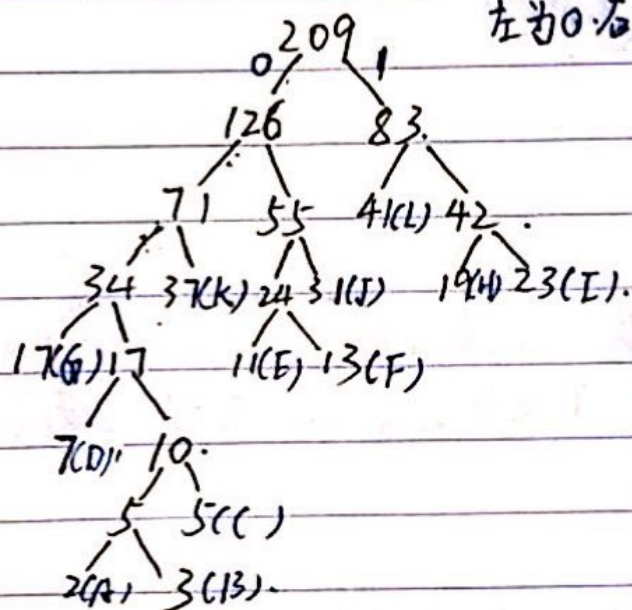
4.

(1) Huffman树与代码

左为0右为1

数据

A	0001100
B	0001101
C	000111
D	00010
E	0100
F	0101
G	0000
H	110
I	111
J	011
K	001
L	10



(2).

由(1)中 Huffman 树:

$$\begin{aligned}
 \text{平均码长} &= \frac{2 \times 4 + 3 \times (37 + 3 + 19 + 23) + 4 \times (17 + 11 + 13) + 5 \times 7 + 6 \times 5 + 7 \times (2 + 3)}{2 + 3 + 5 + 7 + 11 + 13 + 17 + 19 + 23 + 31 + 37 + 41} \\
 &= \frac{676}{209} \\
 &= 3.234
 \end{aligned}$$

5.

(1) 算法设计思想.

通过层序遍历 (BFS) 遍历二叉树的每一层.

统计每一层节点数, 遍历过程中更新更大值.

使用队列实现层序遍历, 对于每一层将所有节点的左右子节点加入队列以处理下一层节点.

(2) 代码

```
int maxwidth(Node* root) { // 计算节点最多的层
    if (root == nullptr) return 0;
    queue<Node*> q; // 使用队列层序遍历
    q.push(root);
    int maxNodes = 0 // 记录最大节点数
    while (!q.empty()) {
        int levelNode = q.size();
        maxNodes = max(maxNodes, levelNode);
        for (int i = 0; i < levelNode; i++) { // 遍历当前层所有节点.
            Node* currentNode = q.front();
            q.pop();
            if (currentNode->left != nullptr) q.push(currentNode->left); // 左子节点入队
            if (currentNode->right != nullptr) q.push(currentNode->right); // 右子节点入队
        }
    }
    return maxNodes;
}
```

(3) 时间复杂度. $O(n)$.

遍历每个节点, 仅入队和出队 1 次, 假设有 n 个节点, 故时间复杂度为 $O(n)$.